GitHub Pages Documentation Deployment with CI/CD Test Gate

This guide provides a step-by-step process for setting up a repository that uses GitHub Actions to automatically build Sphinx documentation from Python docstrings and deploy it to GitHub Pages, only after a full suite of unit tests has successfully passed.

1. Initial Repository Setup

1. **Create Repository:** On GitHub, create a new public repository (e.g., my-calculator-docs). Ensure you check the box to **Initialize this repository with a README.MD file**.
2. **Clone Locally:** Clone the repository and navigate into the directory.
   git clone
   https://github.com/your-username/my-calculator-docs.git
   cd my-calculator-docs

3. **Create Dependency File:** Create a requirements.txt file listing the necessary Python packages.
   # requirements.txt

   pytest

   pre-commit
   sphinx
   sphinx-rtd-theme
   sphinx-autodoc-typehints

4. Create a new virtual env and install requirements
5. (optional) **Install and configure pre-commit**

Like previous lesson

2. Code, Tests, and Documentation Files

Create the following three files in the root of your repository:

2.1. calculator.py (The Source Code)

This file contains the class with standard Python docstrings, which Sphinx will use for API documentation.

```
# calculator.py

class Calculator:
    """
    A simple utility class for basic arithmetic operations.

    This class demonstrates how to use docstrings that are recognized
    by Sphinx's autodoc extension.
    """

    def add(self, a: float, b: float) -> float:
        """
        Adds two numbers.

        :param a: The first number.
        :param b: The second number.
        :return: The sum of a and b.
        """
        return a + b

    def subtract(self, a: float, b: float) -> float:
        """
        Subtracts the second number from the first.

        :param a: The first number (minuend).
        :param b: The second number (subtrahend).
        :return: The difference between a and b.
        """
        return a - b
```

2.2. test_calculator.py (The Unit Tests)

This file ensures the code is working correctly. It must pass before documentation is deployed.

```
# test_calculator.py
import unittest
from calculator import Calculator

class TestCalculator(unittest.TestCase):
    """Tests for the Calculator class."""

    def setUp(self):
        """Set up a new Calculator instance before each test."""
        self.calc = Calculator()

    def test_add_positive(self):
        """Test addition with two positive numbers."""
        self.assertEqual(self.calc.add(5, 3), 8)

    def test_add_negative(self):
        """Test addition with positive and negative numbers."""
        self.assertEqual(self.calc.add(-10, 5), -5)

    def test_subtract_basic(self):
        """Test basic subtraction."""
        self.assertEqual(self.calc.subtract(10, 4), 6)

    def test_subtract_zero(self):
        """Test subtraction resulting in zero."""
        self.assertEqual(self.calc.subtract(5, 5), 0)

if __name__ == '__main__':
    unittest.main()
```

3. Configure Sphinx Documentation

3.1. Initialize Sphinx

Create docs folder (sphinx might not ask you to create one for you!)

mkdir docs

And change dir to it

cd docs

Run the quickstart utility in your repository root. Accept the defaults, but answer **'y'** for "Separate source and build directories (y/n)?" to create the docs/ folder.

sphinx-quickstart

3.2. Update docs/conf.py

Modify the configuration file to enable extensions and allow Sphinx to find your source code (calculator.py).

```
# Edit docs/conf.py

# Add these three lines near the beginning of the file to find the project root
import os
import sys
sys.path.insert(0, os.path.abspath('..'))

# Update the 'extensions' list (around line 30)
extensions = [
    'Sphinx.ext.autodoc',

   'sphinx.ext.autosummary',

    'sphinx.ext.viewcode',

    'sphinx.ext.githubpages',

    'sphinx.ext.coverage',
    'sphinx.ext.napoleon', # Supports numpy and google style docstrings
    'sphinx_rtd_theme',    # Required for the Read the Docs theme
    'sphinx_autodoc_typehints' # Uses Python type hints in the docs
]
```

# Set the HTML theme (around line 170)
html_theme = 'sphinx_rtd_theme'

3.3. Link API in docs/index.rst

Edit your main documentation index to include a link to the new API page.

.. docs/index.rst

calculator documentation

==========

.. automodule:: calculator.calculator

  :members:

.. toctree::

  :maxdepth: 2

  :caption: Contents:

* :ref:`getindex`

* :ref:`modindex`

* :ref:`search`

4. GitHub Actions Workflow (The CI/CD Gate)

Create the workflow file .github/workflows/docs.yml. This file defines two jobs: test and build-and-deploy. The deployment job is dependent on the test job passing.

```yaml
# .github/workflows/docs.yml
name: Build and Deploy Sphinx Docs

# Trigger on pushes to the main/master branch
on:
  push:
    branches:
      - main
      - master


permissions:

  contents: read

  pages: write

  id-token: write


jobs:
  # Job 1: Run Unit Tests (Must pass to proceed)
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.x'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run Unit Tests
        # The '-m unittest' command executes the tests found in test_calculator.py
        run: python -m unittest test_calculator.py
        # If any test fails, this step fails, and the workflow terminates.

  # Job 2: Build and Deploy Documentation
```

```
build-and-deploy:
  runs-on: ubuntu-latest
  needs: test # <--- Ensures this job only runs if the 'test' job succeeded


  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.x'

    - name: Install documentation dependencies
      run: |
        pip install -r requirements.txt

   - name: Sphinx Build

      # Builds the HTML output into the standard _site directory for Pages deployment

      run: sphinx-build -b html docs _site



    - name: Upload artifact

      # Uploads the built documentation as a deployable artifact

      uses: actions/upload-pages-artifact@v3

      with:

        path: '_site'



    - name: Deploy to GitHub Pages

      id: deployment

      # Official GitHub action for deploying the artifact

      uses: actions/deploy-pages@v4
```

5. Final Push and GitHub Pages Configuration

1. **Commit and Push:** Add all created/modified files, commit them, and push to GitHub. This push will trigger the Action.
   git add .
   git commit -m "feat: setup CI/CD with test gate and sphinx documentation"
   git push origin main

2. **Enable GitHub Pages:** Once the Action finishes (it will create a gh-pages branch):
   - Go to your repository **Settings**.
   - Click **Pages** (under Code and automation).
   - Under **Source**, select **Github Actions**

Your documentation will now be live at https://your-username.github.io/my-calculator-docs/. Any future code change requires passing the unit tests before the documentation is updated.